

University of Crete | June 2025

FORTH INSTITUTE OF COMPUTER SCIENCE
Computational Vision & Robotics Laboratory CVRL

POSE ESTIMATION VIA INERTIAL WHEEL ODOMETRY IN AUTONOMOUS VEHICLES

Bachelor Thesis

Dimitrios Asimakopoulos

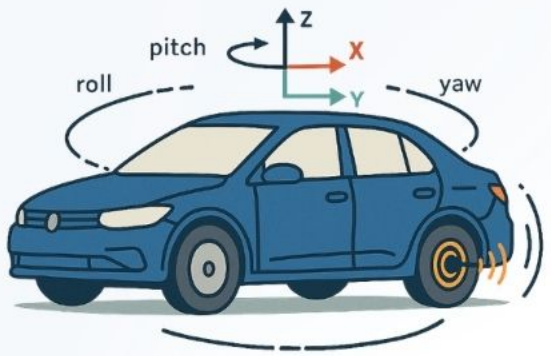
Motivation

- Bosch Future Mobility Challenge
- Accurate vehicle localization is critical for safe autonomous driving
- GPS intermittent; wheel odometry or IMU alone → unbounded drift
- Existing fusion packages (e.g., robot_localization) often opaque, hard to extend
- Need a transparent, modular framework combining odometry, IMU, and vision landmarks
- Simulation (ROS Noetic + Gazebo 11) offers controlled environment



Thesis Overview

- Combines wheel encoders, IMU, and camera-based traffic-sign detector in one ROS/Gazebo module
- Lightweight Extended Kalman Filter named BB EKF (Braking Bad)
- Sign-based updates prevent long-term bias
- Curbs drift, handles sensor noise and dropouts, maintains accurate pose
- Modular ROS nodes + Gazebo plugins → simulation testing & easy transfer to real vehicles
- Simulation trials show significant reduction in cumulative position error vs baseline



Structure

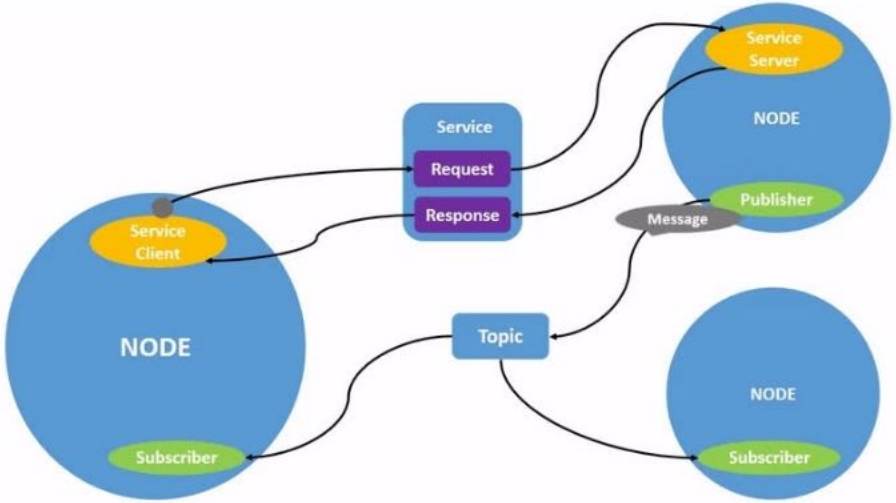
1. **Introduction**
   abstract, overview
2. **Related Work**
   visual odometry, VIO, semantic landmarks, fusion frameworks
3. **Software & Simulation Tools**
   ROS Noetic, Gazebo 11, YOLO integration
4. **Vision & Perception**
   model training, PnP pose estimation, semantic observation pipeline
5. **Inertial & Wheel Odometry**
   plugins, drivers, message formats, kinematic models, noise
6. **Sensor Fusion (EKF)**
   state vector, process/measurement models, Jacobians, tuning
7. **TF & Coordinate Frames**
   frame hierarchy, static/dynamic transforms, validation, conventions
8. **Visualization & Monitoring**
   RViz config, markers, rosbag, rqt, baseline EKF comparison
9. **Evaluation & Results**
   ROS Noetic, Gazebo 11, YOLO integration
10. **Limitations, Future Work & Conclusion**
    model training, PnP pose estimation, semantic observation pipeline

Tools & Software

ROS + GAZEBO + OpenCV

ROS Noetic

- Primary middleware on Ubuntu 20.04
- RViz: interactive 3D interface showing vehicle model, live TF frames, sensor streams, estimated trajectories.
  - Displays ground-truth path, fused odometry, baseline comparison and traffic-sign updates.
  - Enables immediate debugging of coordinate-frame or data-flow issues.



Tools & Software

ROS GAZEBO + OpenCV

Gazebo

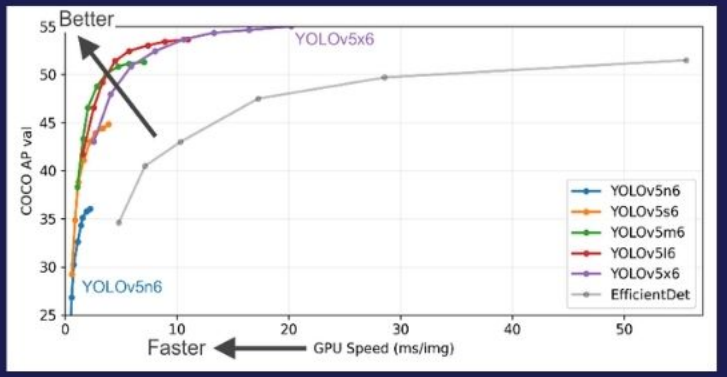
- Ackermann-style car in SDF/URDF with:
  - BNO055 IMU plugin (provides orientation)
  - Custom wheel-encoder plugin (publishes v, ω)
  - Gazebo camera sensor (image feed for YOLO)
- BFMC map usage: city scenario with road markings, crosswalks, roundabouts, parking spaces, traffic-sign models placed at known map coordinates



Tools & Software

YOLOv5 - Nano

- Semantic detection node in Python ROS: subscribes to camera images, runs real-time inference.
- Cropping & undistortion of image ROIs from bounding boxes.
- Outputs high-confidence sign poses published for EKF updates.
- Lightweight model suitable for mid-range hardware.
- Processes 640×640 images



- Custom nine-class dataset (~3,000 images), varied distances/orientations/lighting in Gazebo; 80/20 train-val split; augmentations (brightness, contrast, flips); early stopping + cosine LR schedule → convergence <50 epochs.
- Annotation with LabelImg: tight, consistent boxes normalized to 640×640 to preserve PnP accuracy.





Vision & Perception

Pipeline Overview

1. Camera feed
2. YOLO detection
3. ROI extraction
4. Contour-based corner refinement
5. PnP pose estimation (or fallback)
6. Transform to chassis frame
7. Association against known landmarks
8. EKF update via 2D relative-position measurement

Contour-Based Pose Estimation & Fallback

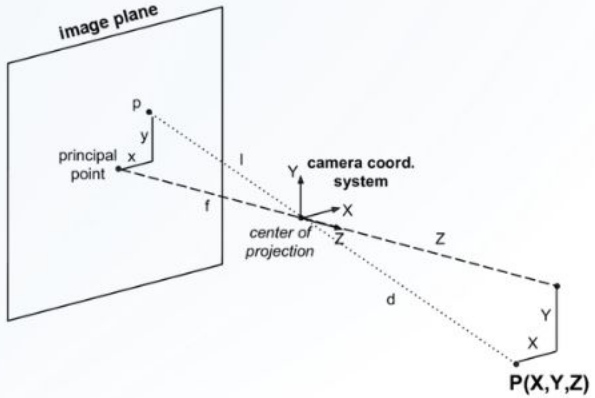
ROI & Preprocessing

- ROI = image patch inside YOLO bbox.
- Convert to HSV → per-class/shape-specific mask (e.g., red border for STOP) + fallback grayscale if needed.
- Morphological closing + Canny edge → extract external contours focusing on border pixels, not background.



Geometric contour selection

- Filter by area; approximate polygon via approxPolyDP.
- Shape checks: rectangle (4 vertices ~90˚), diamond (rotated 4-vertex), triangle (3 vertices), circle ($4\pi \cdot \text{area}/\text{perimeter}^2$).
- If multiple candidates, pick best-fit; else fallback to largest contour;



PnP & fallback

- If 4 valid corners → solvePnPRansac
- If RANSAC fails or insufficient inliers: pinhole model; estimate pose from known sign size + bbox dimensions → compute X,Y via camera intrinsics;

Data Association, Measurement Model & Robustness

Transform to chassis & map

From camera-relative pose → apply static extrinsic → project onto ground plane → (x_est, y_est) in chassis frame

Data association & Mahalanobis gating

- Landmark DB: class ID → known (x_map,y_map). For each detection, compute predicted measurement and innovation.
- Compute Mahalanobis distance and accept smallest distance if below threshold → reduces false matches.

Measurement model & EKF update

- Predict where the known landmark should appear relative to the vehicle given the current pose estimate.
- Compare this predicted relative position to the observed relative position from the sign detection to form the “innovation”.
- Use the current state covariance and the assumed measurement noise to compute the Kalman gain, which balances trust between the prediction and the observation.
- Apply the Kalman gain to the innovation to correct the state estimate.

Robustness checks

- Confidence threshold (≥0.7), temporal consistency (position change ≤0.2m vs last published for same sign), Mahalanobis gating.
- Single-frame buffer: drop stale frames if inference busy → prioritize low latency.
- If no valid sign update, EKF relies solely on wheel & IMU until next detection.

Inertial & Wheel Odometry Overview

Purpose & Role

- Provides continuous short-term motion estimates when external fixes (GPS/vision) aren't available
- Forms the dead-reckoning backbone in between semantic landmark updates



Sensors Employed

- Wheel-encoder plugin (Gazebo) → high-frequency linear & yaw-rate measurements
- BNO055 IMU plugin (Gazebo) → absolute orientation (yaw) at lower rate



Why It Matters for EKF

- High-rate odometry keeps filter predictions accurate between anchor corrections
- IMU heading corrects orientation drift in planar motion

Key Considerations

- Configure realistic noise & update rates to mirror real hardware
- Understand plugin behavior, message formats, and TF conventions

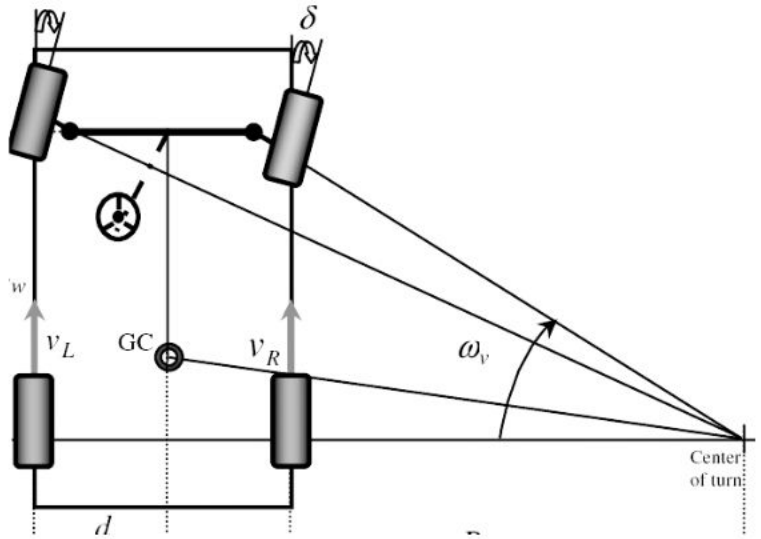
Plugins

Wheel Encoder

- Reads rear-wheel joint velocities + front-steer joint angles from SDF
- Converts to unicycle “twist” (v, ω) under Ackermann kinematics
- Computation Steps (per update at ~100 Hz)
  - Multiply each rear-wheel angular velocity by radius → forward speed v = (v_left + v_right)/2
  - Average front-steer angles → yaw rate ω = v·tan(δ)/wheel_base
  - Dead-zone: zero out v, ω below threshold to suppress jitter at rest
  - Publish nav_msgs/Odometry on /automobile/wheel_encoder/odometry
- Subscriber extracts msg.twist.linear.x and angular.z plus timestamp

BNO055 IMU

- Simulated IMU provides absolute orientation (roll, pitch, yaw) at 10 Hz
- Only yaw influences filter; roll/pitch ignored for planar localization
- Publishes on /automobile/IMU with custom utils_ros/IMU message
- Subscriber callback: read msg.yaw + timestamp



Sensor Fusion (EKF)



Prediction

integrate high-rate odometry & IMU-driven motion model

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \\ v_k \\ \omega_k \end{bmatrix}$$

Correction

measurement updates from odometry, IMU and landmark observations

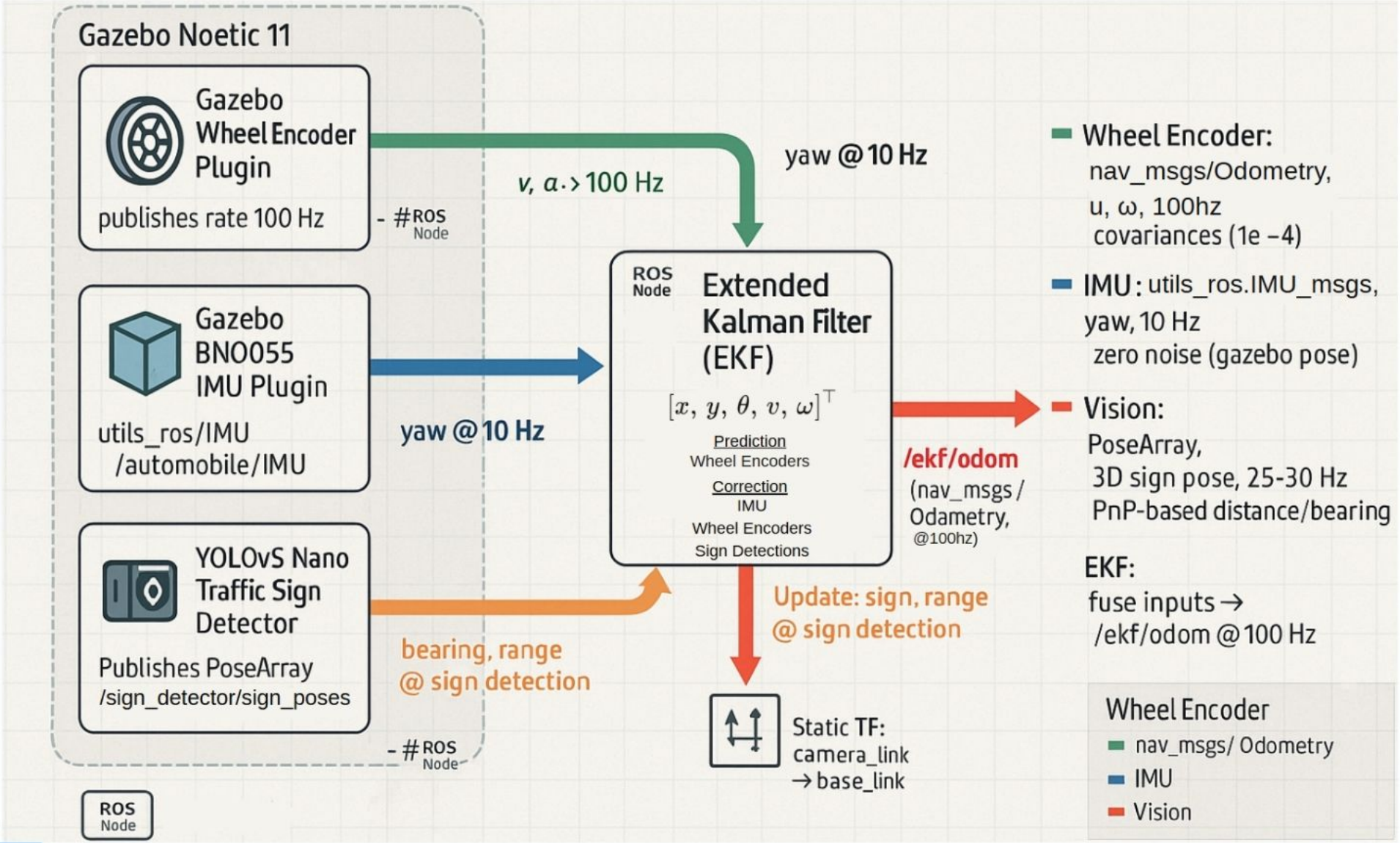


State Initialization & Prediction Model

- Wait until first reliable pose (ground-truth or known initial) arrives; initialize position (x, y) and heading from that pose
- Set moderate initial uncertainty on position, heading, and larger uncertainty on velocities
- Use last-known forward speed and yaw rate to predict next pose
- Model small uncertainty in acceleration to capture unmodeled dynamics
- Propagate covariance to reflect growing uncertainty until next measurement
- Frequent, lightweight prediction keeps filter “alive” between updates, so measurement corrections stay small and stable.

Measurement Updates

- Compare readings of forward speed and yaw rate from the encoder plugin to predicted state's speed/yaw rate, correct state accordingly
- Use absolute heading measurement from IMU plugin to correct accumulated heading drift from odometry alone
- Odometry and IMU updates arrive at different rates; each triggers a predict+update cycle if filter is initialized
- For each detected sign class, compare estimated relative position against expected map positions
- Once matched, fuse this relative-position observation into EKF: adjust position and heading to align vehicle pose so that predicted landmark range matches measurement
- Semantic fixes act as periodic "resets," keeping drift in check; combined with odometry/IMU, yield a robust pose estimate even over long runs.


Gazebo Noetic 11
Gazebo Wheel Encoder Plugin
publishes rate 100 Hz
- #ROS Node
v, a -> 100 Hz
yaw @ 10 Hz
Gazebo BNO055 IMU Plugin
utils_ros/IMU
/automobile/IMU
yaw @ 10 Hz
YOLOvS Nano Traffic Sign Detector
Publishes PoseArray
/sign_detector/sign_poses
bearing, range
@ sign detection
- #ROS Node
ROS Node
Extended Kalman Filter (EKF)
$[x, y, \theta, v, \omega]^\top$
Prediction
Wheel Encoders
Correction
IMU
Wheel Encoders
Sign Detections
/ekf/odom
(nav_msgs / Odometry, @100hz)
Update: sign, range
@ sign detection
Static TF:
camera_link
→ base_link
Wheel Encoder:
nav_msgs/Odometry,
u, ω, 100hz
covariances (1e −4)
IMU: utils_ros.IMU_msgs,
yaw, 10 Hz
zero noise (gazebo pose)
Vision:
PoseArray,
3D sign pose, 25-30 Hz
PnP-based distance/bearing
EKF:
fuse inputs →
/ekf/odom @ 100 Hz
Wheel Encoder
nav_msgs/ Odometry
IMU
Vision
ROS Node

Frame Hierarchy & Roles

world
gazebo's reference

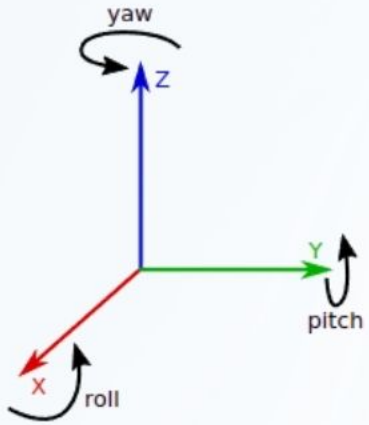
odom
local odometry frame

map
EKF global frame

chassis::link
vehicle base link

camera_link
camera frame

- Use rosrun tf view_frames + RViz TF display to confirm: No cycles (loop-free) and complete coverage (all frames reachable from chassis::link/map)
- Coordinate Convention (ENU, right-handed): X forward, Y left, Z up.



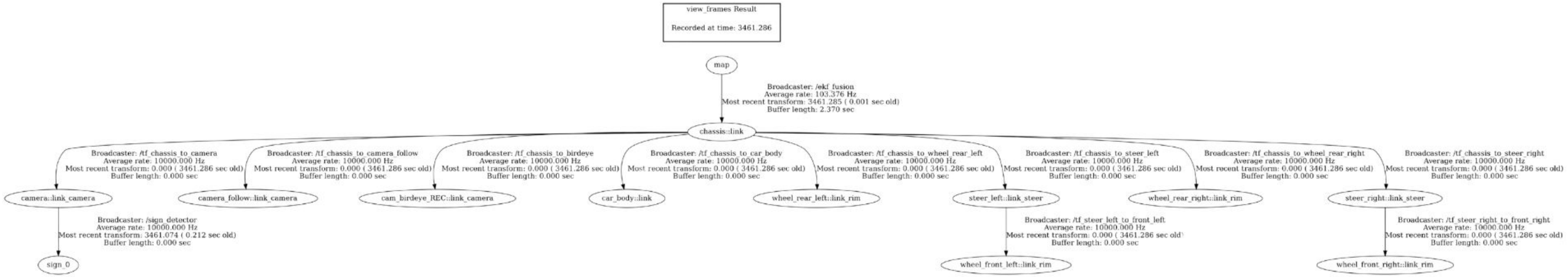
Static vs Dynamic

Static Transformations

- Defined once in launch for chassis::link→sensor mounts and steer joints → front wheel rims
- Published at high rate but unchanged in pose.
- Ensures zero-lag parent→child relations for camera, IMU link, etc.

Dynamic Transformations

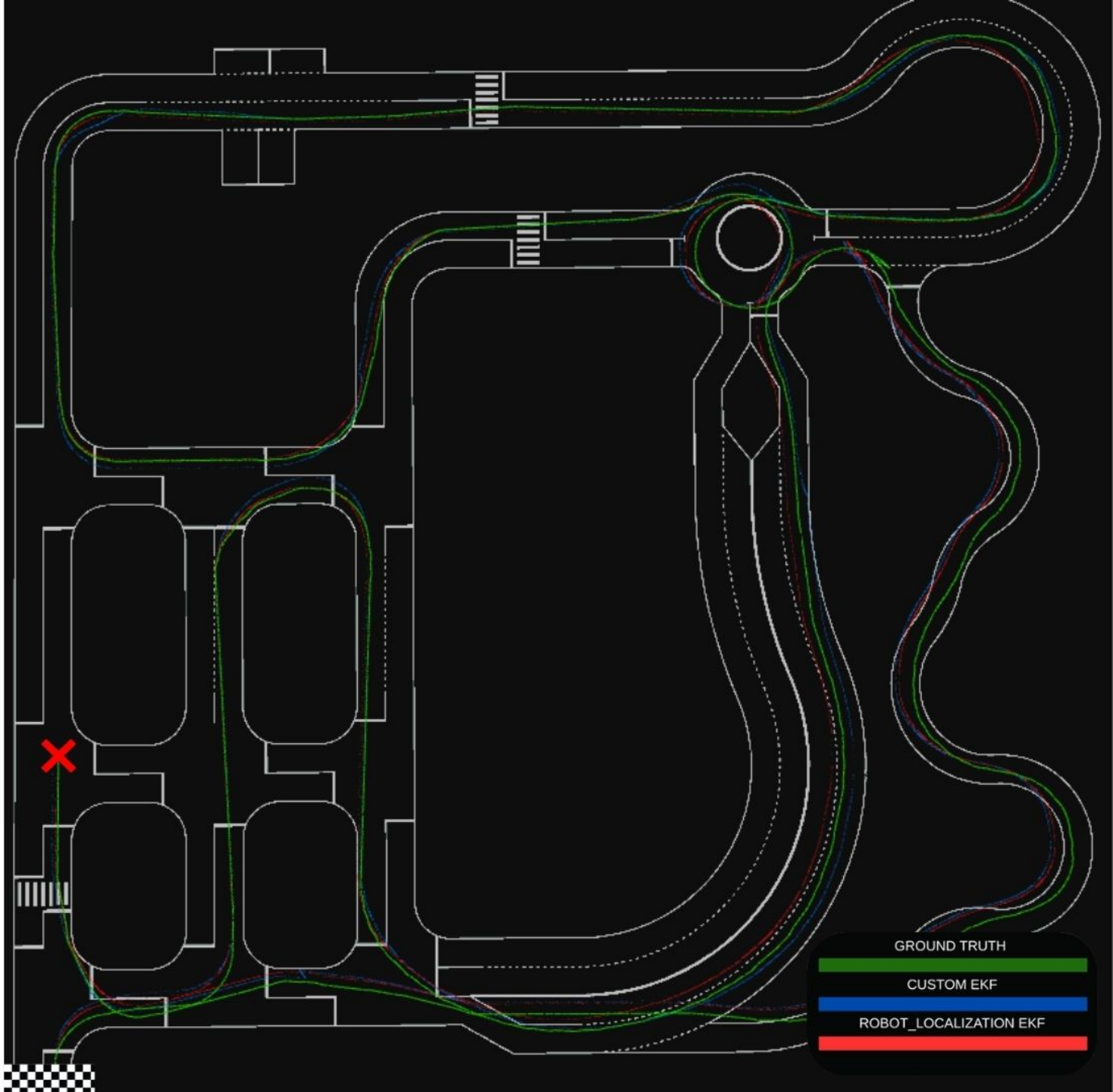
- EKF output: map→chassis::link stamped at each fused estimate (~100 Hz).
- Use TransformStamped in EKF node to broadcast vehicle pose in map frame.

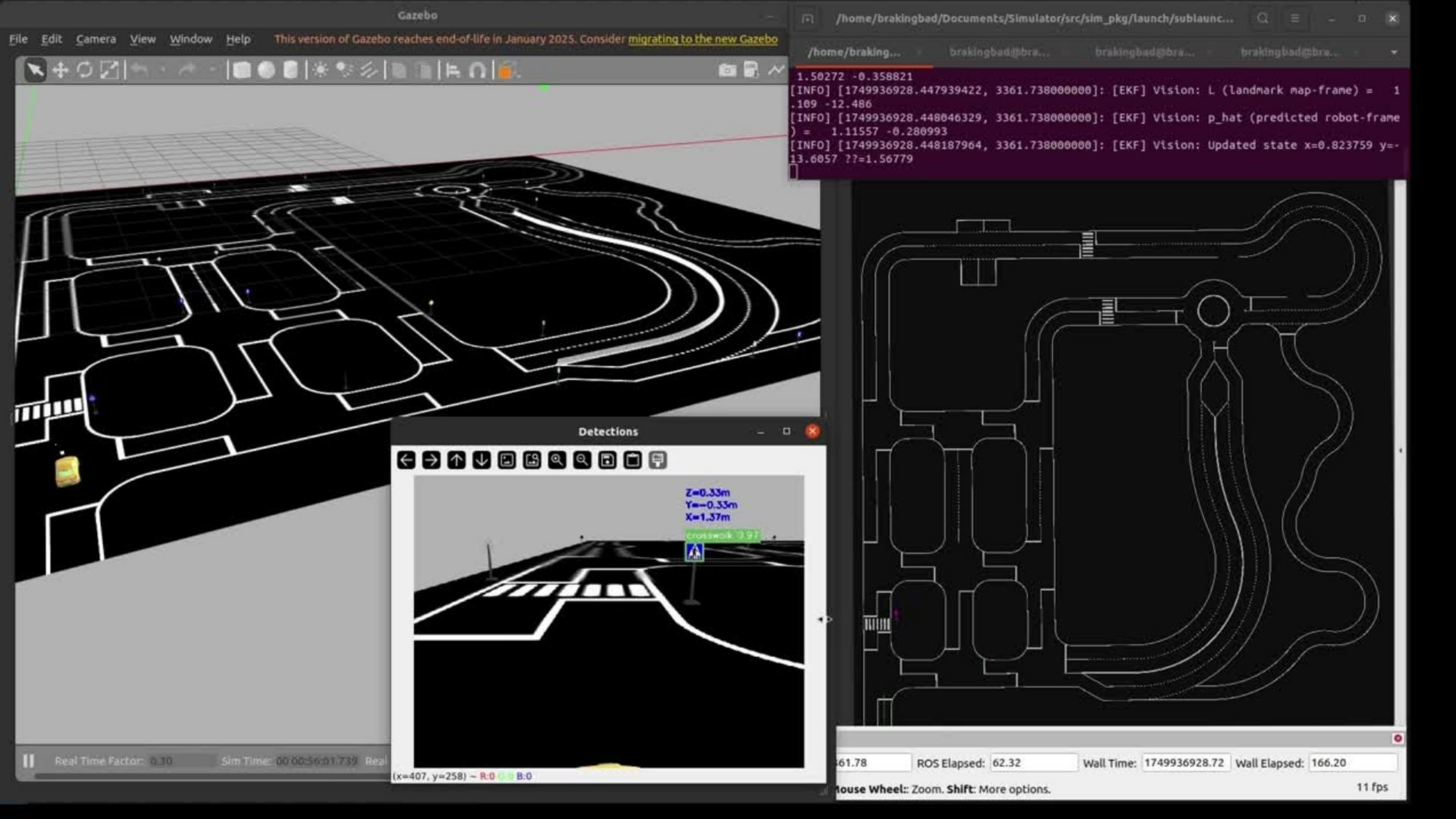


Visualisation

- Fixed Frame = map: All displays plotted in global coordinates.
- Ground Truth (green): /ground_truth_path.
- VIWO EKF Path (blue): /ekf/path.
- Baseline EKF (red): /robot_localization/path.
- Display detected sign poses (/sign_detector/sign_poses) as arrows.





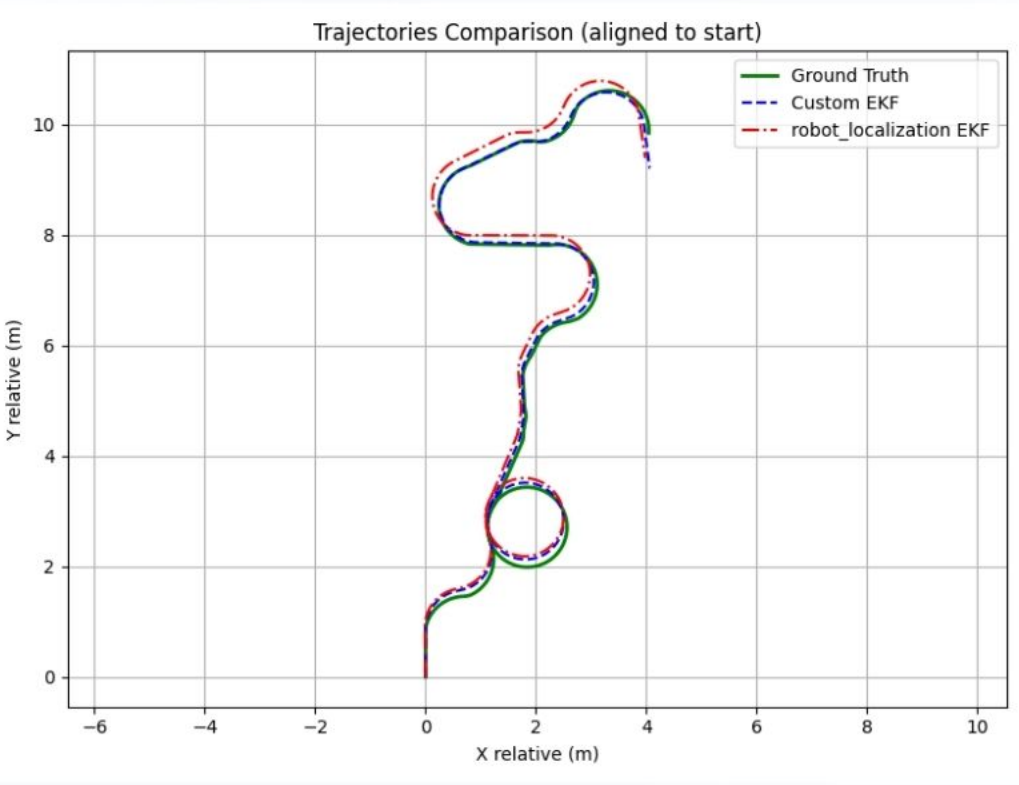

Gazebo
File Edit Camera View Window Help
This version of Gazebo reaches end-of-life in January 2025. Consider migrating to the new Gazebo
Real Time Factor: 0.30
Sim Time: 00 00:56:01.739
/home/brakingbad/Documents/Simulator/src/sim_pkg/launch/sublaunc...
/home/braking...
brakingbad@bra...
brakingbad@bra...
brakingbad@bra...
1.50272 -0.358821
[INFO] [1749936928.447939422, 3361.738000000]: [EKF] Vision: L (landmark map-frame) = 1
.109 -12.486
[INFO] [1749936928.448046329, 3361.738000000]: [EKF] Vision: p_hat (predicted robot-frame
) = 1.11557 -0.280993
[INFO] [1749936928.448187964, 3361.738000000]: [EKF] Vision: Updated state x=0.823759 y=-
13.6057 ??=1.56779
Detections
Z=0.33m
Y=-0.33m
X=1.37m
crosswalk 0.97
(x=407, y=258) ~ R:0 G:0 B:0
ROS Elapsed: 62.32
Wall Time: 1749936928.72
Wall Elapsed: 166.20
Mouse Wheel:: Zoom. Shift: More options.
11 fps

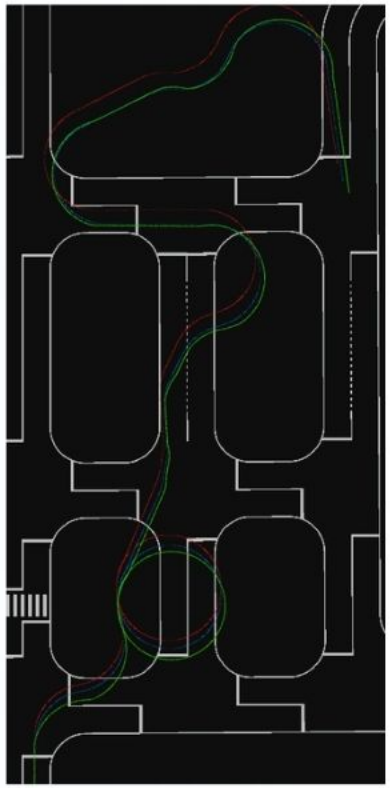
Evaluation Methodology & Sensor Characterization

- Record ROS bags with topics: /ground_truth_path, /ekf/path, /robot_localization/path

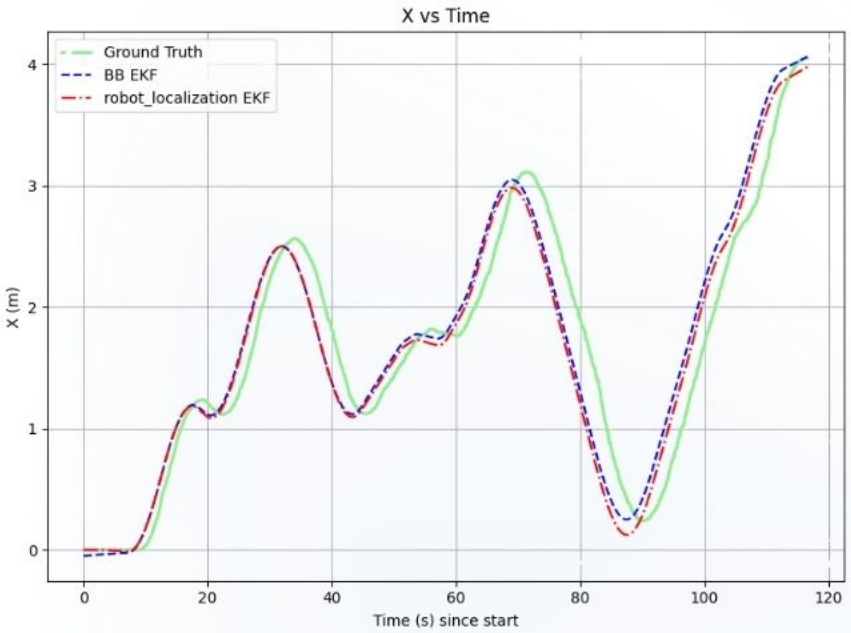
- **evaluate.py**
  - extract time-aligned DataFrames of (time, x, y, z, yaw).
  - align estimates to ground truth within 0.05s tolerance.
  - position error = Euclidean distance between estimate and GT.
  - yaw error = absolute minimal angular difference.

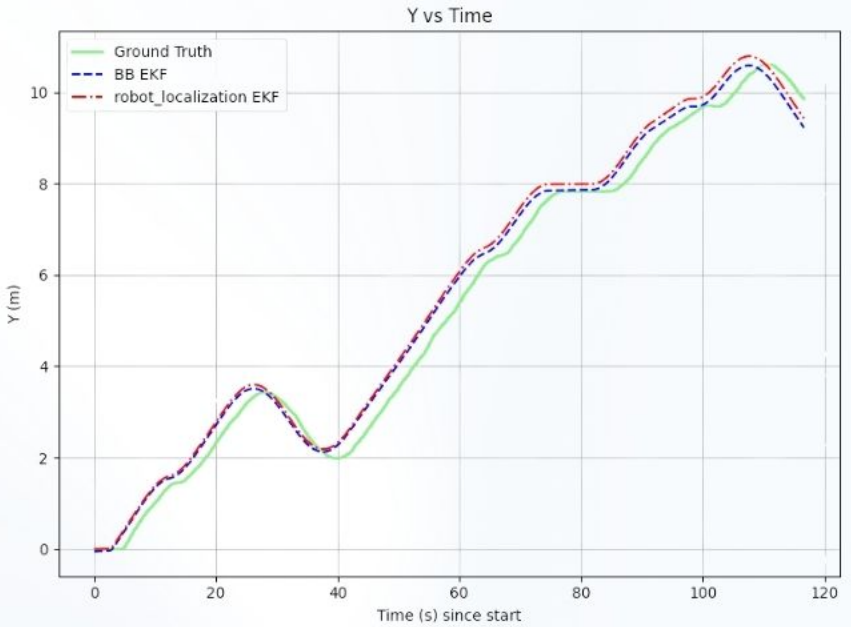
- **characterize_sensors.py**
  - buffers data
  - interpolates GT at sensor timestamps
  - computes residuals and suggests empirical R_odom, R_imu.
  - ensures EKF noise covariances are data-driven, not purely heuristic.

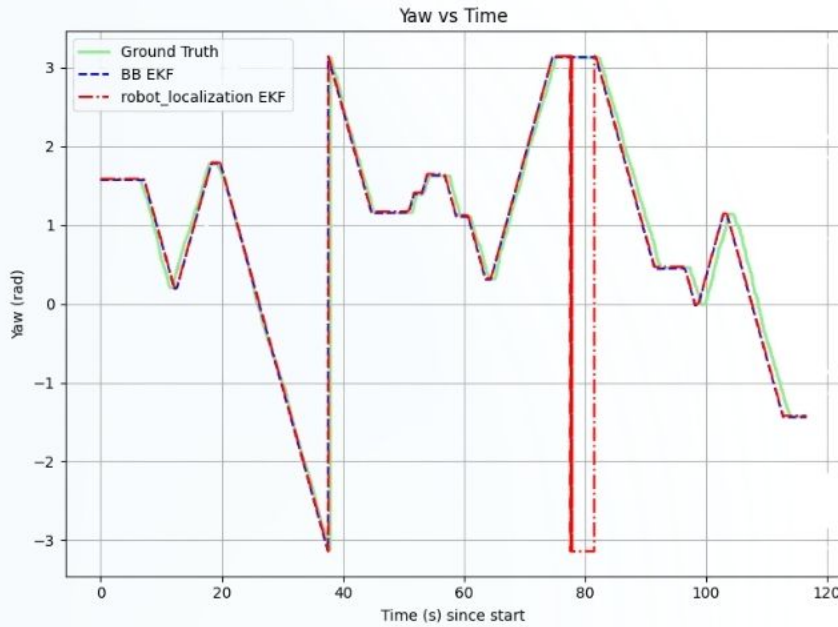




X,Y,YAW vs TIME





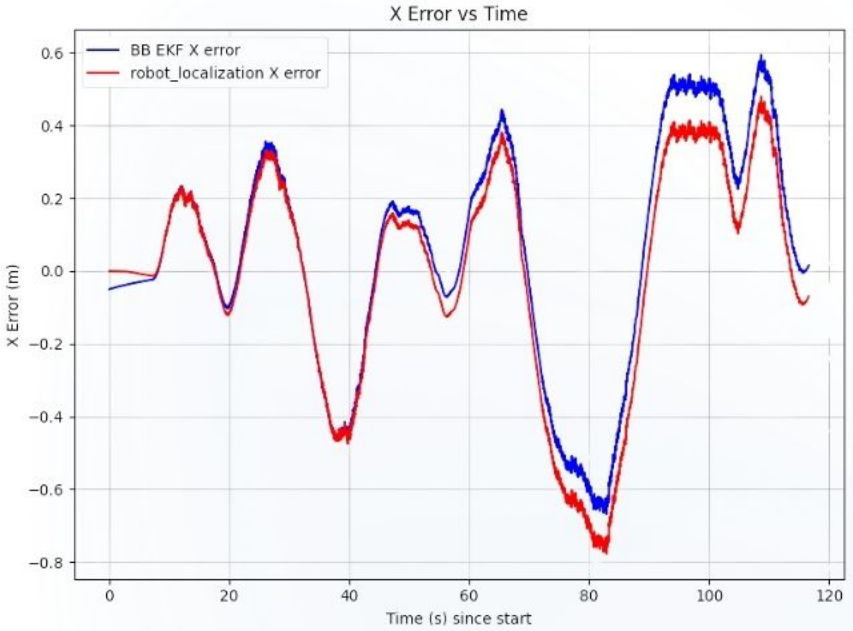


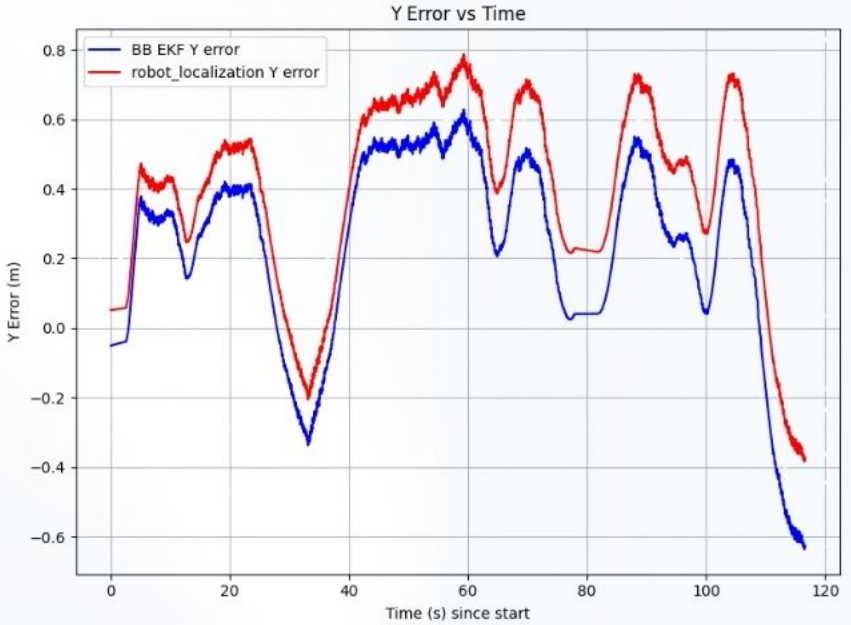
BB EKF tracks GT X closely, with smaller cumulative offset.

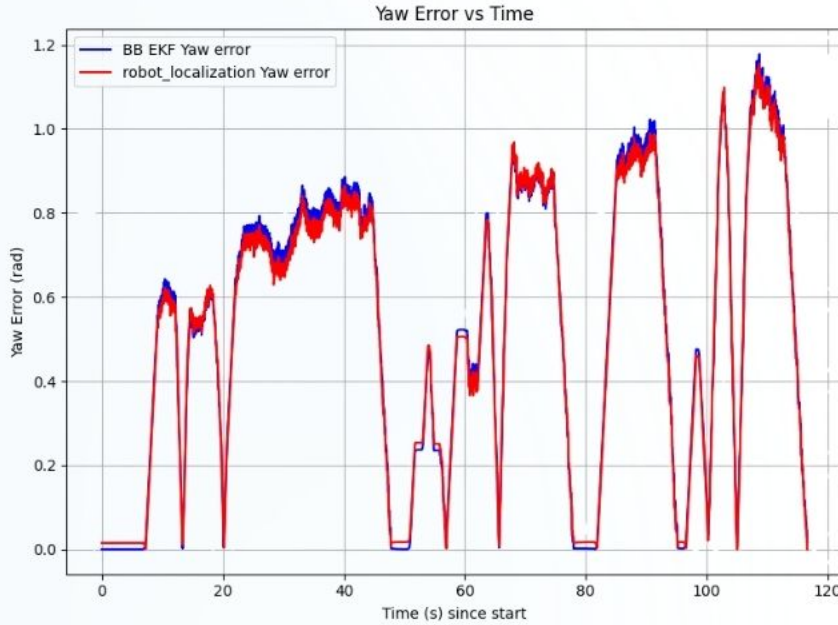
BB EKF maintains lateral position nearer to ground truth

BB EKF may exhibit occasional small peaks before correction

ERROR vs TIME







BB EKF performs slightly worse possibly due to PnP miscalculations.

BB EKF maintains lower lateral error peaks and deeper corrective dips

Both EKFs track heading error almost identically - vision has minimal impact

Conclusion

This project develops a modular ROS/Gazebo framework that fuses wheel odometry, IMU data, and traffic-sign vision detections via a custom EKF to achieve robust, drift-reduced vehicle localization.

Limitations

- Simulation-Only Evaluation means simplified noise & dynamics.
- IMU: perfect orientation in Gazebo vs. real drift/bias.
- Wheel Encoders: no slip model vs. actual terrain-induced errors.
- Landmark updates require pre-mapped sign locations.
- No loop-closure or SLAM for unknown landmarks.
- Real-world sensor imperfections likely further degrade performance.

Future Work

- Algorithmic Enhancements (e.g. Unscented KF, Sliding-window).
- Additional Sensors (LiDAR, GPS).
- Real-World Testing.
- Incorporate other semantic cues (lane markings, traffic lights).
- Integrate with obstacle avoidance or planning.
- Migrate to ROS 2 for robustness.



University of Crete | June 2025

FORTH
INSTITUTE OF COMPUTER SCIENCE
Computational Vision & Robotics Laboratory CVRL

THANK YOU!